

技术报告：长上下文场景中大模型自动数据标注

项目	信息
队伍名称	卷不动
成员	肖傲文, 崔超, 杨崇海

摘要

针对基于 Qwen3-4B 大模型的长上下文场景自动化数据标注任务，本文聚焦 32K 上下文窗口资源受限、多类型标注任务适配性差、模型标注一致性弱以及代码类生成任务难以核验等行业痛点开展研究，主要工作包括：

- 首先完成全赛道任务数据精细化预处理，通过**混淆样本过滤**、**多维度类别标签标注**等方式优化样本质量，剔除易造成模型判断偏差的干扰数据。
- 针对原生 Nanobot 智能体面向长会话设计的记忆管理机制不适配批量数据标注场景的缺陷，重构智能体运行逻辑，取消无效历史会话信息读取，搭建**评估 - 推理双阶段运行架构**，构建**反馈驱动型记忆更新机制**，结合强化学习思想实现提示规则与示例策略的动态迭代优化，同时构建难样例库辅助高质量 few-shot。
- 在长上下文高效利用层面，创新性提出分任务自适应少样本示例遴选方案：情绪检测任务（任务5）**构建难例样本缓存库**，以短样本优先策略扩充窗口内有效样本数量；主题分类任务（任务6）采用**同源类别优先选取策略**，结合正反例规则明确判定边界；百科问答（任务7）任务搭建多领域标签体系，实现同领域示例动态匹配筛选，最大限度挖掘上下文信息价值。
- 设计差异化标注执行流程，常规标注任务采用单轮规则约束式标注范式（任务1-7），Triton 算子代码生成任务（任务8）则搭建五轮迭代四层级闭环验证体系，依次完成语法校验、结构核验、测试用例生成与数值结果对齐，依托历史错误反馈驱动模型自主修正输出内容。
- 针对任务5，任务6，尝试采用无参GRPO进行了prompt优化，整体成绩提升0.2-0.3

实测结果表明，技术方案可显著提升各类任务标注精度，有效解决长上下文资源浪费、模型判错率高、生成结果不可靠等问题，具备较强的通用性与实用性，可为大模型长上下文自动化数据标注工程落地提供可行的技术参考。

一、赛题理解

基于Qwen3-4B大语言模型，采用上下文（In-context Learning, ICL）范式开展**自动化数据标注**任务研究。本赛题的主要挑战在于：

- 长上下文的高效利用挑战**：32K 上下文窗口容量有限，而部分任务（如 Task7、Task5）训练示例数量庞大（数千个），如何在窗口内筛选“高价值示例”（提升标注准确性），同时避免信息冗余（浪费 token 资源）成为关键；
- 任务多样性的适配挑战**：分类、生成、代码四大类任务的标注逻辑差异显著——分类任务需明确标签边界，生成任务需保证输出完整性，代码任务需满足语法与数值正确性，需设计统一且可扩展的标注框架；

3. **标注一致性与准确性平衡挑战**：无人工干预场景下，模型易受输入表述、示例顺序等因素影响导致输出波动，需通过规则约束、示例筛选等方式强化一致性；同时部分任务（如 Task6）存在“主题匹配”与“语义相似度”的易混淆点，需通过指令设计降低误判率；
4. **特殊任务的正确性验证挑战**：Task8（代码生成）需同时满足“语法正确”“结构合规”“数值一致”三大要求，传统单轮标注无法保证正确性，需设计闭环验证机制。

二、数据分析与数据预处理

本章对赛题任务进行分析，赛题一、三相对简单，任务8为代码生成，因此不做特殊分析，主要针对赛题二、四至七进行分析。

1、任务二

任务二为统计句子中名词和动词的数量，但是给出的样例中存在容易导致大模型迷惑信息，例如下面两个例子，standing在句子中的句式等基本一致，单在第一个例子中任务是动词，第二个句子中认为不是动词。

```
"id": "openseek-2-4b7a93143ed74da797f90ef6e2dbc822",
: "Sentence: 'an elephant standing in some tall grass next to the bushes'. Count the number of verbs in this sentence.",
: [
    "1"
],

"id": "openseek-2-b6de31e8398c4abeb6d58011ad5239ad",
: "Sentence: 'A skier in red standing at the top of a mountain'. Count the number of verbs in this sentence.",
: [
    "0"
],
```

处理策略：

1、打标签：针对每个问题分别构造标签,句子中是否出现ing，分别筛选是统计Nouns还是verbs，用于prompt筛选

2、过滤混淆样本：对于句子中含有ing，统计verb，结果是0的标记为useless,不放入few-shot

处理后的数据如下：

```
{
  "id": "openseek-2-f94477005563444181ce3154ec053752",
  : "Sentence: 'an gray iron direction sign a window and a brick building'. Count the number of nouns in this sentence.",
  : [
    "5"
  ],
  : "nouns,ing",
  : false
},
```

2、任务四

任务四也相对明确，但直接采用大模型在部分例子中易出错，尤其是在字符串中出现“,”时，如下例子：

```
{
  "id": "openseek-4-3bb7b896295842d0a3d2b8a6f98d6b8c",
  "input": "['p', 'that.', 'o']",
  "output": [
    "pthat.o"
  ]
},
{
  "id": "openseek-4-cb042df46ae94fc49087e85f31f7725e",
  "input": "['B', 'g', 'it', 'j', 'x', 'a', 'which,', 'e', 'o', 'b', 'the', 'M', 'j']",
  "output": [
    "Bgitjxawhich,eObtheMj"
  ]
},
```

处理策略：
在prompt中具体体现相应的例子

3、任务五

任务五为判断句子是否为“Sad”或者“Not sad”，两者占比约为4：6，其中有部分句子较为明确，但部分句子表达较为模糊，尤其是部分陈述句等，例如：

```
# My house isnt always a mess but when it is, its when i have friends coming
#mess #serious #twitter #problem #like #friends #filthy #home <label>Sad</label>
# Shoutout to the drunk man on the bus who pissed in a bottle and on the seats
#grim <label>Sad</label>
```

除此之外，任务5里面含有一些@开头的ID信息，可能导致混乱。

处理策略：

- 1、利用正则表达式去除@数据
- 2、利用迭代的思路，首先运行一次，过滤出易错样本，作为样本库，在下一次运行中从样本库中进行提取。

4、任务六

任务六为判断句子是否属于同一个genre，首先，部分genre在例子和表述任务中没有出现，出现的genre包括：

genre	数量	占比
telephone	1199	21.8%
government	1097	19.9%

genre	数量	占比
travel	1085	19.7%
fiction	1085	19.7%
slate	1034	18.8%

处理策略：

- 1、打标签：每个例子均打上对应的genre标签，在推理中分别从对应的genre中选择标注为Y和N的样本

5、任务七

任务7为问答，整体数据类型较多且难度较大，涵盖历史、文化、体育等多个类型。

处理策略：

- 1、打标签：每个例子均打上对应的类别标签，在推理中分别从对应的标签中选择样本

标签后的样本数据分布如下（前1000个）：

Tag	数量	占比
other	140	14.0%
geography	124	12.4%
celebrity_people	116	11.6%
title_quote	100	10.0%
history	88	8.8%
literature	77	7.7%
science	61	6.1%
animal	55	5.5%
food_dessert	39	3.9%
daily_life	33	3.3%
politics	30	3.0%
military	23	2.3%
anatomy_medical	23	2.3%
bible_religion	22	2.2%
language_trans	18	1.8%
number_trivia	18	1.8%
proverb	17	1.7%
auto_sports	16	1.6%

Tag	数量	占比
总计	1000	100%

三、整体思路

1. 设计思路

我们希望在标注任务中加入多轮对话，反思等机制，并希望模型能够调用部分代码工具等，nanobot[1]等智能体框架，提供了比较好的基础，对tools，MCP，Skill等做了比较好的集成。并且通过测试提取经验（history+dream能力），更新memory等能力，实现标注过程的自演进[2]。

同时，我们也认识到现有的nanobot是针对长对话逐渐积累经验的，对于数据标注这一类任务并不试用。主要体现在：

Nanobot 原有的 Memory 管理逻辑是为**长会话场景**设计的，前提是前后对话逻辑保持一致：

1. 当上下文过长时，把内容压入 `history` 存储
2. 由 `Dream` 模块定时扫描 `history`，更新用户信息和 Memory 文件
3. 按 `cursor` 指针位置，分批次扫描 `history` 和 `Dream` 内容

在当前的数据标注场景中，原有机制会失效：

1. **多主题历史会话干扰**：尤其是小模型，容易被过往多主题的对话带偏，影响当前任务判断
2. **短会话 / 多用户场景更新滞后**：多用户、短对话、高并发场景下，history加载机制混乱，Memory 无法被及时更新
3. **内部任务无法迭代优化**：像数据标注、故障定位这类需要持续沉淀经验的任务，没法通过这套逻辑实现迭代体现

因此提出了新方案：

1. **流程拆分：验证 + 更新**：用验证环节驱动 Memory 更新，通过持续迭代提升准确率
2. **更新逻辑改为“反馈驱动”**：不再只依赖上下文，而是用任务的反馈结果来更新 history
3. **用 RL 逻辑更新 Memory 策略**：把 Memory 里的规则、few-shot 示例、prompt 等看作“策略（policy）”，用强化学习[3]的思路迭代优化

我们对nanobot进行了代码优化，在每次构建prompt中不读取历史信息，避免上下文混乱，同时采用evaluation+inference过程，整体的设计思路如下图所示：

1、在Evaluation阶段，在测试数据上运行：

- 1、当任务报错时显式调用nanobot history接口，写入history文件；
- 2、设计奖励函数，构造经验回放池；
- 3、同时对执行失败的任务构建难样例库，用于构建高质量few-shot。

2、显式调用Dream过程，更新memory，或者采用GRPO的方式进行prompt优化

3、执行推理过程

通过上述策略，一方面构建难样例库（例如在任务5中，见具体实现方案），另一方面，在一定程度上实现了对上下文和模型引导的自动优化，例如，在任务5中，通过模型总结出了如下规则：

Sad could also means angry, discouraged, guilty, awkward, criticize, complain and so on

在任务7中，通过模型总结出了：

2. Extract key information from the Clue: proper nouns, years, locations, people, abbreviations, allusions, features and foreign language content.

2. 具体任务优化策略

2.1 分任务优化

任务5 (Tweet悲伤情绪检测)：在指令设计上，我们通过结构化推理规则引导模型综合考虑推文文本、hashtag和emoji，避免单一特征的误判；针对示例数量远超上下文容量的问题，我们**预构建代表性示例缓存**，按token长度升序排序，在30K上下文窗口内最大化有效示例数量（约50个），通过短样本优先策略实现信息密度最大化；该任务采用单轮稳定标注框架，通过统一的输出格式约束（<label> 标签包裹）确保标注一致性。

任务6 (MNLI同Genre分类)：针对主题相关性判断极易与语义相似度混淆的难点，我们设计了包含10组正反例对照的显式规则指令，明确告知模型“即使观点相反、事实冲突，只要同主题就标Y”这一核心判断标准；在超长上下文构造上，我们采用**按Genre分组的示例选择策略**，**优先选取与待标注样本同Genre的示例（最多50个）**，再补充其他Genre的代表性样例，确保在30K窗口内既保证同分布参考又覆盖多样性；该任务的一致性通过严格的规则注入和多线程独立标注实现。

任务7 (Jeopardy答案生成)：在提示策略上，我们设计了四步结构化推理流程（领域定位→关键信息提取→百科知识匹配→答案校验），引导模型进行系统性思考而非盲目猜测；针对数千个训练样本远超30K上下文容量的问题，我们创新性地引入17个领域分类标签体系，**预先对所有样本进行自动领域分类**，**标注时先对待标注样本进行在线分类**，再动态选取相同领域的100个最相关示例构建上下文；这种预分类+动态匹配的策略极大提升了超长上下文的信息密度和相关性，同时领域标签体系本身具有良好的可扩展性。

任务8 (Kernel算子生成)：针对Triton kernel代码生成对正确性要求极高的特点，我们设计了***5轮迭代验证***的完整闭环提示策略，每轮包含四个严格验证步骤：(1) Python语法编译检查；(2) AST解析与基本结构验证（检查@triton.jit装饰器、wrapper函数）；(3) 自动生成PyTorch参考实现与测试用例；(4) 实际运行kernel并与参考结果进行torch.allclose数值校验；在上下文利用上，每轮prompt不仅包含任务描述和参考示例，还动态注入最近2次失败尝试的代码片段和具体错误反馈，实现持续学习式的自我修正；在多轮交互中，我们通过保留错误反馈的历史上下文，实现了迭代过程中的信息累积和错误模式学习，直到测试通过或达到最大迭代次数。

任务	核心策略	示例选择方式	特殊机制
Task 5	情绪分类	预构建代表性缓存	按长度排序最大化示例数
Task 6	主题匹配	按 Genre 分组选择	详细规则 + 正反例约束
Task 7	百科问答	按领域 Tag 匹配	预分类 + 结构化推理
Task 8	代码生成	全量示例（小窗口）	多轮迭代 + 语法验证

2.2 投票策略

对每个任务分别执行n次推理（任务8设定 n=1），选择频次最高的作为最终输出结果。

四、具体实现方案

1. 整体标注流程

我们采用统一的标注框架，针对各任务特点进行定制化扩展：

1. **任务自适应示例选择**：每个任务采用最适合其数据分布的示例选择策略
2. **显式规则约束**：通过结构化的规则描述强化模型行为一致性
3. **长上下文高效利用**：在 32K 上下文窗口内最大化有效示例数量
4. **多线程并行标注**：通过线程池提升整体标注吞吐量

2.具体分任务实现

2.1 任务5：Tweet 悲伤情绪检测（预构建缓存+示例）

2.1.1 构建典型部分

```
CLASS SadnessCacheSelector:
    FUNCTION run():
        cache = load_existing_cache()
        all_samples = load_train_set()

        // 32线程并行筛选零样本预测错误的难例
        FOR EACH sample IN all_samples WITH 32 THREADS:

            // 用零样本预测，判断是否是难例
            prediction = annotate_sample(sample.input, task_id=5)
            IF prediction != sample.output:
                // 格式化后加入缓存
                cache.append(format(sample))
                save_cache(cache)

        cache = deduplicate(cache) // 按输入内容去重
        save_cache(cache)

END
```

2.1.2 标注部分

```
// ===== 预处理：预先构建代表性缓存 =====
FUNCTION build_sadness_cache():
    selector = Task5SadnessSelector(args)
    selector.run()
    // 保存到：./data_processed/task5_sadness_cache

// ===== 标注时动态加载 =====
IF task_id == 5:
    // 自动加载预构建的缓存示例
```

```

examples = select_examples_from_cache(
    "./data_processed/task5_sadness_cache",
    task_desc, text, args, max_samples=50 // 50个示例 ~25K tokens
)
rules = RULES[5] // 多步推理规则
use_tools = False
clean_mentions(text) // 去除@
clean_mentions(examples)
bot, result = annotate_nanobot(task_desc, text, 5, examples, rules,
use_tools)
RETURN result

```

2.2 任务6: MNLI 同 Genre 分类 (按 Genre 分组+示例)

```

FUNCTION select_examples_by_genre(icl_examples, task_desc, text, args,
n_per_genre=50):
    target_genre = regex_extract(r'Genre:\s*(\w+)', text)
    groups = group_by_genre(icl_examples)

    selected = []
    token_count = 0
    target_length = 25_000

    // 第一步: 优先选同Genre的50个示例
    IF target_genre IN groups:
        FOR ex IN groups[target_genre][:50]:
            IF token_count + estimate_tokens(ex) < target_length:
                selected.append(format_example(ex))
                token_count += estimate_tokens(ex)

    // 第二步: 补充其他Genre的代表性示例 (每类10个)
    FOR genre, ex_list IN groups.items():
        IF genre == target_genre:
            CONTINUE
        FOR ex IN ex_list[:min(10, n_per_genre//5)]:
            IF token_count + estimate_tokens(ex) < target_length:
                selected.append(format_example(ex))
                token_count += estimate_tokens(ex)

    RETURN JOIN(selected)

// ===== 标注调用 =====
IF task_id == 6:
    examples = select_examples_by_genre(icl_examples, task_desc, text, args, 50)
    rules = TASK6_ANNOTATION_RULES // 包含10组正反例
    use_tools = False
    bot, result = annotate_nanobot(task_desc, text, 6, examples, rules,
use_tools)
    RETURN result

```

2.3 任务7: Jeopardy 答案生成 (领域 Tag 匹配+100示例)

2.3.1 预处理部分

```
TAGS = [auto_sports, food, military, geography, celebrity,
        literature, religion, science, history, proverb, language,
        animal, number_trivia, title_quote, medical, daily_life, politics,
        other]

FUNCTION tag_all_samples():
    train_samples = load_train_set()

    // 多线程并行分类打标签
    WITH ThreadPoolExecutor(16):
        tagged_samples = parallel_map(classify_one, train_samples)

    save(tagged_samples, output_file)
FUNCTION classify_one(sample):
    prompt = f"""
        Classify this Jeopardy question into one tag: {TAGS}
        Input: {sample.input}
        Output ONLY the tag wrapped in <label>
        """

    result = nanobot.run(prompt)
    tag = extract_from_label(result)

    RETURN (sample, tag) IF tag IN TAGS ELSE (sample, "other")
```

2.3.2 推理部分

```
// ===== 标注时按Tag选择 =====
IF task_id == 7:
    max_samples = 100
    IF tag EXISTS:
        // 按Tag选择相同领域的示例
        examples = select_examples_by_tag(icl_examples, task_desc, text, args,
tag, max_samples)
    ELSE:
        // 无Tag时自动选择最多100个, 按长度过滤
        examples = select_n_examples(icl_examples, task_desc, text, args,
max_samples)

    rules = RULES[7] // 4步结构化推理
    use_tools = False
    bot, result = annotate_nanobot(task_desc, text, 7, examples, rules,
use_tools)
    RETURN result
```

2.4 任务8: Kernel 算子生成 (零样本+5轮四步闭环验证)

```
// 关键: examples = "" 且 rules = "" // 完全零样本!
// 完全靠迭代验证机制保证正确性 (来自 method.py 第800-939行)

FUNCTION annotate_nanobot_task8(task_desc, text, task_id=8, examples="",
rules="", use_tools=False):
    max_iterations = 5
    feedback_history = []
    bot = None

    FOR iteration FROM 1 TO max_iterations:
        bot = Nanobot.from_config("./config.json", use_tools=False)

        // 构建含最近2次失败反馈的prompt
        feedback_section = format_last_2_attempts(feedback_history)

        prompt = f"""
            # Task: Triton Kernel Generation
            {task_desc}

            ## Core Requirements
            1. Generate complete, runnable Python code with Triton kernels
            2. Must include both @triton.jit kernel function AND Python wrapper
            3. Include necessary imports: import torch, import triton, import
triton.language as tl
            4. Follow exact style of reference examples (if any)
            5. Correctly handle BLOCK_SIZE, pointers, masks, offsets

            {feedback_section}

            ## Text to Annotate
            {text}
            """

        code_candidate = extract_answer(bot.run(prompt))
        IF code_candidate IS None:
            feedback_history.append("", "Failed to extract code")
            CONTINUE

        // ===== 四步验证闭环 =====
        // Step 1: Python语法编译检查
        TRY:
            compile(code_candidate, '<string>', 'exec')
        EXCEPT SyntaxError e:
            feedback_history.append((code_candidate, f"Syntax error: {e}"))
            CONTINUE

        // Step 2: AST结构验证 + 导入检查
        success, output = _validation_code(code_candidate)
        // 验证内容:
        // - parse AST successfully
        // - check @triton.jit decorator exists
        // - check wrapper function exists
        IF NOT success:
```

```

        feedback_history.append((code_candidate, output))
        CONTINUE

// Step 3: 自动生成PyTorch参考实现与测试用例
// 重试最多3次
test_code = generate_test_case_with_retry(task_desc, text,
code_candidate, bot, max_retry=3)
IF test_code IS None OR len(test_code) < 10:
    feedback_history.append((code_candidate, "Failed to generate tests"))
    CONTINUE

// Step 4: 实际运行kernel并与PyTorch参考数值校验
// 使用 torch.allclose(rtol=1e-3, atol=1e-3) 断言
full_code = code_candidate + "\n\n# ===== TEST CODE =====\n" + test_code
test_success, test_output = execute_code_safe(full_code, timeout=30)

IF test_success AND "✅ ALL TESTS PASSED" in test_output:
    RETURN bot, code_candidate // 验证通过, 提前返回
ELSE:
    feedback_history.append((code_candidate, test_output[:500]))
    CONTINUE

// 5轮仍失败, 返回最后一次尝试
RETURN bot, (feedback_history[-1][0] IF feedback_history ELSE "")

```

3. GRPO (Task-5 和 Task-6)

同时, 对于任务5和任务6, 我们尝试基于强化学习算法进一步优化分类任务的提示词。针对分类任务, 我们对 Training-Free GRPO 框架进行了针对性的适配, 以下为Task-5的具体实现步骤如下 (Task-6类似) :

a. 数据预处理

基于 四、具体实现方案 中的任务5和6获取分类错误的数据 + 部分分类正确数据, 将其制作为json训练集。格式如下:

```

[
  {
    "prompt": "Sentence 1: But the insurers are fighting an irresistible force. Sentence 2: The new rule would alter how current reporting requirements in Medicare hospitals work. Genre: government.",
    "answer": "N"
  },
  {
    "prompt": "Sentence 1: 1 billion in 2002 and increased to $5. Sentence 2: There is no need for any staff in emergency rooms to screen patients. Genre: government.",
    "answer": "N"
  },
  {
    "prompt": "Sentence 1: At the national level, there are a total of 575 wrestlers classed in six divisions according to their win-loss ratio in the annual tournaments. Sentence 2: There are about 60 wrestlers at the national level. Genre: travel.",
    "answer": "Y"
  }
]

```

```
},  
    ...  
]
```

b. 多轨迹采样

对于每一个训练样本，我们构建如下的 Prompt 输入给大模型：

```
<|ROLLOUT_START|>  
\# 任务经验  
[这里插入当前经验库中的所有经验条目]  
  
\# 任务要求  
你是一个情感分析专家，给定一条推文，判断作者是否感到悲伤。  
你只能输出 "Sad" 或 "Not sad"，并且必须说明你的判断理由。  
  
\# 推文内容  
{input_tweet}  
<|ROLLOUT_END|>
```

基于该 Prompt，我们对同一个样本并行采样 **G=5** 条不同的输出轨迹。这 5 条轨迹代表了模型在当前经验下，尝试的 5 种不同的判断策略。

c. 奖励函数设计

我们设计了双维度的奖励函数，全面评估每个轨迹的质量：

- **正确性奖励 (Core Reward) :**

如果模型输出的分类结果与真实标签一致，奖励 +1.0 分。

如果分类错误，奖励 0.0 分。

- **合规性奖励 (Format Reward) :**

检查模型的推理过程是否同时提到了文本内容、emoji 和 hashtag 的分析，避免模型只依赖单一信号。

如果模型的理由中体现了“综合判断”的逻辑，额外奖励 +0.2 分，引导模型符合任务的约束要求。

d. 语义优势提取

这是整个算法最核心的一步。我们将 5 条轨迹及其对应的奖励分数输入给大模型，让它进行元推理：

```
<TRAJECTORY_START>  
下面是针对同一条推文的5个不同的判断结果，其中有的正确，有的错误。  
请你分析它们之间的差异，总结出一条通用的、可以指导未来判断的经验。  
这条经验应该说明：为了正确判断悲伤情绪，我们应该注意什么？  
  
轨迹1: ... 奖励: 1.0  
轨迹2: ... 奖励: 0.0  
...  
  
<TRAJECTORY_END>
```

通过这个过程，模型会自动从成败案例中提炼出通用的经验。例如，针对本次任务，模型会自动总结出如下的经验：

- 1. 当推文中包含 sad、depressed、heartbroken 等词汇，或 悲伤类 emoji 时，大概率为 Sad，但必须结合上下文验证，不能仅靠关键词匹配。
- 2. 不能仅通过 emoji 判断情绪，例如用户可能使用表情表达夸张的喜悦或反讽，必须阅读完整文本确认语境。
- 3. 需关注 hashtag 的语义，#SadDay 通常指向悲伤，但 #NotSad 或 #HappySad 则表示相反的含义。
- 4. 对于包含 @用户的对话类推文，要注意这是两人的交流，情绪可能是针对对方的话，不能孤立地看单句。
- 5. 遇到反讽表达时，例如 "So happy I missed the exam"，要注意字面意思和实际情绪的反转。

e. 经验库优化

提取出新的经验后，我们会让控制器 LLM 对经验库进行维护：

- Add**：如果新经验是全新的、有价值的，就添加到库中。
- Delete**：如果旧经验被证明是错误的，或者与新经验冲突，就删除它。
- Modify**：如果新旧经验相似，就合并它们，生成更通用、更准确的经验。
- Keep**：如果没有变化，就保留原有经验。

这个过程保证了经验库始终保持精简、准确，不会随着迭代变得冗余。

f. 迭代与推理

我们重复上述过程 **3 个 Epoch**，使用仅 500 个标注样本进行训练。每一轮迭代后，经验库都会变得更强大，引导模型做出更准确的判断。在最终的推理阶段，我们只需要将训练好的经验库插入到 Prompt 中，就可以让模型直接对新的推文进行分类，无需任何其他修改。

e.效果

任务5	效果
人为定义system_prompt	11.47
grpo, TEMPERATURE=0	11.58
grpo, TEMPERATURE=1	11.6

任务6	效果
人为定义system_prompt	11.25
grpo, TEMPERATURE=0	11.35
grpo, TEMPERATURE=1	11.5

五、结果分析

5.1 结果

由于提交次数限制，以及中间多次尝试，仅提供部分具有代表性的消融实现结果：

策略	提升效果
任务2：按照tag进行过滤	+0.5
任务5：构建代表性样例以筛选prompt	+1
任务5：Prompt自动优化	+0.2
任务5: GRPO	+0.13
任务6：同Genre分类	+2
任务6：GRPO	+0.25
任务7：构建tag并进行过滤	+0.3
任务7：Prompt自动优化	+0.1
任务8：构建代码生成+多轮验证机制	+2

5.2 讨论与局限性

基于Context 优化的模型演进策略，被称为Fast Learning，尽管业界存在相关研究[2] [3] [4]，但通常没有讨论在长上下文情况下的应用，以及与智能体结合的策略；本方案在一定程度借鉴了相关的思路。

在构建解决方案初期，我们希望从“统一”的角度实现数据标注，即将不同的“经验”写入到同一个memory文件中，实现同一套逻辑，大模型在不同任务中自动识别不同规则和经验，避免各个任务独立运行，在这个基础上实现自演进。但在实际应用中，发现各个任务间干扰较大，其可能由于模型的上下文推理能力不足导致。

由于4B模型推理和总结能力较弱，经过验证，4B模型不具备识别和调用skill能力，且tools能力有限，因此将nanobot中的skill、系统tools等进行了屏蔽，避免上下文污染。除此，对history总结后的信息进行了人工分析，然后写入了分任务的rules文件中，在当前具体运行中实际没有完全实现自演进能力，还需要部分人工介入，后续借助更加强大的模型希望能够实现整个流程的自动化。

参考文献

[1]<https://github.com/HKUDS/nanobot>

[2]Learning, Fast and Slow: Towards LLMs That Adapt Continually,<https://arxiv.org/pdf/2605.12484>

[3]Training-Free Group Relative Policy Optimization, [arXiv:2510.08191](https://arxiv.org/abs/2510.08191)

[4]GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning, [arXiv:2507.19457](https://arxiv.org/abs/2507.19457)

附录

代码

提交代码，具体部署和执行过程见README.md

参赛人员

肖傲文, 476756210@qq.com, 男, 马上消费, 算法工程师, 曾获得首届全国先进计算技术创新大赛一等奖、第十五届中国研究生电子设计竞赛西南分赛区三等奖、第九届重庆市科慧研究生创新创业大赛三等奖等竞赛奖励;

崔超, cuichao03@inspur.com, 男, 浪潮通信, 算法工程师, 曾获得首届全国先进计算技术创新大赛一等奖、山东省第二届数据应用创新创业大赛二等奖、世界人工智能大会-人工智能应用大赛三等奖、第七届世界智能大会算法赛三等奖等竞赛奖励。

杨崇海, ychai918@163.com, 紫光展锐, 算法工程师, 曾获得重庆市研究生电子设计大赛三等奖

硬件环境

硬件环境基于比赛方提供的算力资源，包括两张NPU 910B(64G)显卡，基于flagscale，采用pipeline并行模式部署模型，32并发，启用KVCache。具体配置如下(模型启动命令见README.md):

```
root@f9145b39470e:~# more /workspace/examples/qwen3/conf/serve.yaml
defaults:
- _self_
- serve: 4b

experiment:
  exp_name: qwen3_4b
  exp_dir: outputs/${experiment.exp_name}
  task:
    type: serve
    backend: vllm
  runner:
    hostfile: null
  deploy:
    use_fs_serve: false
  envs:
    CUDA_VISIBLE_DEVICES: 0,1
    CUDA_DEVICE_MAX_CONNECTIONS: 1

action: run

hydra:
  run:
    dir: ${experiment.exp_dir}/hydra

root@f9145b39470e:~# more /workspace/examples/qwen3/conf/serve.yaml
defaults:
- _self_
- serve: 4b

experiment:
```

```
exp_name: qwen3_4b
exp_dir: outputs/${experiment.exp_name}
task:
  type: serve
  backend: vllm
runner:
  hostfile: null
  deploy:
    use_fs_serve: false
envs:
  CUDA_VISIBLE_DEVICES: 0,1
  CUDA_DEVICE_MAX_CONNECTIONS: 1

action: run

hydra:
  run:
    dir: ${experiment.exp_dir}/hydra
```